CHEAT SHEET · GROUPE 1

FastAPI en pratique

Installation · routes · Pydantic · dépendances · tests · async | Sources : fastapi.tiangolo.com, docs.pydantic.dev, Real Python

INSTALLER & LANCER

```
python -m venv .venv
.venv\Scripts\activate      # Windows
pip install "fastapi[standard]"

fastapi dev main.py         # dev + reload
fastapi run main.py         # production
uvicorn main:app --reload --port 8000
```

Docs auto : `/docs` · `/redoc` · `/openapi.json`

APPLICATION MINIMALE

```
from fastapi import FastAPI

app = FastAPI(
    title="API Tâches",
    version="1.0.0",
)

@app.get("/")
def home():
    return {"message": "Hello FastAPI"}

@app.get("/health", tags=["ops"])
def health():
    return {"status": "ok"}
```

ROUTES & MÉTHODES

```
@app.get("/tasks")          # lire
@app.post("/tasks")         # créer
@app.get("/tasks/{id}")     # lire un
@app.put("/tasks/{id}")     # remplacer
@app.patch("/tasks/{id}")   # modifier
@app.delete("/tasks/{id}") # supprimer
```

Retourner un `dict`, une `list` ou un modèle Pydantic produit du JSON.

PARAMÈTRES

```
from typing import Annotated
from fastapi import Path, Query

@app.get("/tasks")
def list_tasks(
    done: Annotated[bool | None, Query()] = None,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    offset: Annotated[int, Query(ge=0)] = 0,
):
    ...

@app.get("/tasks/{task_id}")
def get_task(
    task_id: Annotated[int, Path(ge=1)],
):
    ...
```

- **Path** : segment d'URL, obligatoire.
- **Query** : `?limit=20&offset=0`.
- **Body** : modèle Pydantic (JSON).
- **Header / Cookie** : via `Annotated`.

DÉPENDANCES (Depends)

```
from fastapi import Depends

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def get_current_user(
    token: str = Depends(oauth2_scheme),
    db=Depends(get_db),
):
    ...

@app.get("/me")
def me(user=Depends(get_current_user)):
    return user
```

Factorise auth, DB, pagination. Testable via `app.dependency_overrides`.

ROUTERS

```
# routers/tasks.py
from fastapi import APIRouter
router = APIRouter(
    prefix="/api/v1/tasks",
    tags=["tasks"],
)

@router.get("")
def list_tasks(): ...

# main.py
app.include_router(tasks.router)
```

ASYNC : LE BON RÉFLEXE

```
@app.get("/sync")
def sync_endpoint():
    # threadpool FastAPI
    return requests.get(url).json()

@app.get("/async")
async def async_endpoint():
    async with httpx.AsyncClient() as c:
        r = await c.get(url)
    return r.json()

@app.get("/bad")
async def bad():
    # DANGER : bloque la boucle
    return requests.get(url).json()
```

I/O → `async def` + `httpx` / `asyncpg`. CPU → `def` ou process pool.

MODÈLES PYDANTIC

```python
from pydantic import BaseModel, Field

class TaskCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    due_date: date | None = None

class TaskUpdate(BaseModel):
    title: str | None = None
    done: bool | None = None

class Task(TaskCreate):
    id: int
    done: bool = False
```

Séparer **entrée**, **mise à jour** et **sortie**.

RÉPONSES & ERREURS

```python
@app.post("/tasks", response_model=Task,
          status_code=201)
def create_task(payload: TaskCreate):
    ...

raise HTTPException(
    status_code=404,
    detail="Tâche introuvable",
    headers={"X-Error-Code": "task_not_found"},
)
```

`response_model` filtre les champs exposés. Validation → **422** automatique.

TESTS

```python
# tests/test_tasks.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_create():
    r = client.post("/api/v1/tasks",
                    json={"title": "Test"})
    assert r.status_code == 201
    assert r.json()["title"] == "Test"

def test_validation():
    r = client.post("/api/v1/tasks",
                    json={"title": ""})
    assert r.status_code == 422
```

Lancer : `pytest -q`

CONFIG & SÉCURITÉ

```python
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    jwt_secret: str
    cors_origins: list[str] = []

settings = Settings()  # lit .env + env
```

```python
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

CHECKLIST PROJET

- Routers par domaine, `main.py` mince.
- Modèles Pydantic d'entrée/sortie séparés.
- Ne jamais exposer directement l'ORM.
- Variables d'environnement, pas de secrets en dur.
- CORS explicite, HTTPS, rate limiting.
- Tests cas nominaux + erreurs + auth.
- Logs corrélés, health check, métriques.
- Dockerfile + CI/CD + documentation OpenAPI.

ERREURS FRÉQUENTES

- `async def` + bibliothèque bloquante → tout ralentit.
- Oublier `await` → coroutine non exécutée.
- Exposer le modèle ORM → fuite de champs sensibles.
- `allow_origins=["*"]` en production.
- Pas de pagination → réponses énormes.
- Mélanger `PUT` partiel et `PATCH`.